

Smart Grocery Cart Assistant

*Built on top of the JioMart Retail Product Catalog, [sourced]
(<https://www.kaggle.com/datasets/satyamsundaram/jiomart-products-dataset>) from
Kaggle*

Objective

Create a Gradio-based AI assistant that recommends a weekly grocery shopping cart based on the user's dietary needs and preferences. The app uses Retrieval-Augmented Generation (RAG) on a structured JioMart product dataset, generating explainable, structured results that are visually rendered like a real shopping cart.

Part 1 — User App (Essential Features)

Inputs

- **Grocery Needs:** Free-form text input
(e.g., "high protein, no besan or curd")

Output

- **Suggestion:** A natural language explanation of what's recommended and why
- **Shopping Cart:** Structured as a visual gallery containing:
 - Product Name
 - Quantity
 - Price
 - Product Image

Part 2 — Visual Experience

Visual Cart (Gradio `gr.Gallery`)

The cart is rendered in a grid format using `gr.Gallery` , where each item includes:

- Product image (`image_url`)
- Caption text with:
 - `item_name`
 - `Qty.{quantity}`
 - `₹total_price`

This provides a realistic and user-friendly shopping experience.

Part 3 — Developer-Facing Advanced Settings

Shown under a collapsible **LLM Settings (Advanced)** section in the UI.

Model & Generation Settings

- **Model Selector** (+additional 2 models from Groq after looking at benchmarks and other things):
 - LLaMA 3.3 70B (`llama-3.3-70b-versatile`)
- **Temperature Slider**: Adjustable between 0.0 and 1.5

These settings allow developers to tune how deterministic or creative the model's responses are.

Backend Setup

Vector Store and RAG

- **Embedding Model:** LLaMA 3.2 (3B) via Ollama
- **Vector Database:** ChromaDB
- **RAG Workflow:**
 1. Load product catalog using `CSVLoader`
 2. Parse and clean fields into `Document` objects with metadata
 3. Generate embeddings and index documents
 4. Perform similarity search based on user preferences
 5. Use an LLM to generate structured output parsed via `PydanticOutputParser`

Sample CSV Schema Mapping

Field in CSV	Mapped Use
title	item_name and quantity (parsed from name)
discountedPrice	unit_price
filename	image_url
subType	sub_category
type	category

Advanced:

Developer features:

- Chunk Size, Chunk Overlap
- Top K (similarity search)
- MMR, Hybrid search

User-features:

- Budget constraints (given a budget, do this for me)
- Integrate with diet plan for 1 week, and ingredients needed

```
In [1]: import gradio as gr
import pandas as pd
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings
from langchain_community.document_loaders import CSVLoader
from langchain.chat_models import init_chat_model
from langchain_openai import OpenAI
from langchain.prompts import ChatPromptTemplate
from langchain.output_parsers import PydanticOutputParser
from langchain.schema.output_parser import OutputParserException
from langchain_core.documents import Document
from pydantic import BaseModel
from typing import List
from dotenv import load_dotenv

import os
import warnings
import getpass
import shutil
```

```
In [2]: warnings.filterwarnings('ignore')
load_dotenv(override=True)
```

Out[2]: True

```
In [3]: #Check for Groq API Key
if "GROQ_API_KEY" not in os.environ:
    os.environ["GROQ_API_KEY"] = getpass.getpass("GROQ API Key: ")
```

```
In [4]: #Set Hugging Face token (as we will be using some of Hugging Face's
functionalities
from huggingface_hub.hf_api import HfFolder
HfFolder.save_token(os.environ["HF_TOKEN"])
```

```
In [5]: # Load and preprocess data
loader = CSVLoader(file_path="jiomart_products_database.csv",
source_column="title")
documents_raw = loader.load()
```

```
In [6]: # Convert page_content string to dict and build metadata
documents = []
for doc in documents_raw:
    try:
        row_data = dict(
            line.split(":", 1) for line in doc.page_content.split("\n")
        if ":" in line
        )
        row_data = {k.strip(): v.strip() for k, v in row_data.items()}

        page_text = f"Name: {row_data.get('title', '')} | Sub-type:
{row_data.get('subType', '')} | Type: {row_data.get('type', '')} |
Price: {row_data.get('discountedPrice', 0)} | Image:
{row_data.get('filename', '')}"
        metadata = {
            "category": row_data.get("type", ""),
            "sub_category": row_data.get("subType", "")
        }

        documents.append(Document(page_content=page_text,
metadata=metadata))

    except Exception as e:
        print("Skipping row due to error:", e)
```

```
In [7]: len(documents)
```

Out[7]: 5672

Chat Model

```
In [8]: from langchain.chat_models import init_chat_model

model_name = "openai/gpt-oss-120b"
llm = init_chat_model(model_name, model_provider="groq")
```



```
In [ ]: # Create vector store using all available product documents
if os.path.exists("./Chroma_langchain_db"):
    print("Existing Chroma database found. Reusing it.")
else:
    print("Creating new Chroma database.")

# Create vector store using the first 100 documents
vector_store_chroma = Chroma.from_documents(
    documents=documents[:100],
    embedding=embeddings_model,
    collection_name="advanced_rag",
    persist_directory="./Chroma_langchain_db"
)

# Convert vector store into a retriever
retriever = vector_store_chroma.as_retriever(
    search_kwargs={"k": 10}
)
```

Existing Chroma database found. Reusing it.

```
In [13]: from pydantic import BaseModel, Field
from typing import List

class GroceryItem(BaseModel):
    item_name: str
    price: float
    quantity: int
    image_url: str

class GroceryOutput(BaseModel):
    reasoning: str
    items: List[GroceryItem]
```

```
In [14]: parser = PydanticOutputParser(pydantic_object=GroceryOutput)
```

```
In [15]: from langchain_core.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate.from_template("""
You are a helpful grocery shopping assistant.

User preferences:
{preferences}

Retrieved product context:
{context}

{format_instructions}

Instructions:
1. Recommend only products that appear in the retrieved product context.
2. Do not invent product names, prices, quantities, or image paths.
3. Select products that best match the user's preferences.
4. If the retrieved products are not a good match, return an empty items
list and explain why in reasoning.
5. Keep the reasoning concise.
""")
```

```
In [16]: # Groq model mapping
# The original Llama/Qwen model IDs used in the assignment have been
# deprecated on Groq.
model_name_map = {
    "Llama-3.3 (70B) (Groq)": "openai/gpt-oss-120b",
    "Llama-3.1 (8B) (Groq)": "openai/gpt-oss-20b",
    "Qwen-3 (32B) (Groq)": "qwen/qwen3.6-27b"
}

model_choices = list(model_name_map.keys())
```

```
In [17]: from langchain.chat_models import init_chat_model

def get_llm(model_choice, temperature):
    model_name = model_name_map[model_choice]
    return init_chat_model(
        model_name,
        model_provider="groq",
        temperature=temperature
    )
```

```
In [18]: def generate_cart(model_choice, user_input):
    context_docs = retriever.invoke(user_input["preferences"])

    if not context_docs:
        return GroceryOutput(
            reasoning="No relevant products were retrieved for the given
preferences.",
            items=[]
        )

    relevant_text = "\n".join(
        f"Product {i + 1}: {doc.page_content}"
        for i, doc in enumerate(context_docs)
    )

    llm = get_llm(
        model_choice,
        temperature=user_input.get("temperature", 0.7)
    )

    prompt = prompt_template.format(
        preferences=user_input["preferences"],
        context=relevant_text,
        format_instructions=parser.get_format_instructions()
    )

    try:
        output = llm.invoke(prompt)
        result = parser.parse(output.content)
        return result
    except OutputParserException:
        return GroceryOutput(
            reasoning="The model returned an invalid structured
response. Please try again.",
            items=[]
        )
    except Exception as e:
        return GroceryOutput(
            reasoning=f"Unable to generate the shopping cart: {str(e)}",
            items=[]
        )
```

```

In [19]: # User Interface
def gradio_interface(preferences, model_choice, temperature):
    user_input = {
        "preferences": preferences,
        "model_choice": model_choice,
        "temperature": temperature
    }

    result = generate_cart(model_choice, user_input)

    if not isinstance(result, GroceryOutput):
        return str(result), []

    gallery_items = [
        (
            item.image_url,
            f"{item.item_name}\nQty: {item.quantity}\n₹{item.quantity *
item.price}"
        )
        for item in result.items
        if item.image_url
    ]

    return result.reasoning, gallery_items

demo = gr.Interface(
    fn=gradio_interface,
    inputs=[
        gr.Textbox(
            label="Describe your grocery needs (e.g., 'high protein, no
besan or curd')"
        ),
        gr.Dropdown(
            label="Model",
            choices=model_choices,
            value=model_choices[0]
        ),
        gr.Slider(
            minimum=0.0,
            maximum=1.5,
            value=0.7,
            step=0.1,
            label="Temperature"
        )
    ],
    outputs=[
        gr.Textbox(label="Considerations"),
        gr.Gallery(label="Shopping Cart", columns=3, height="auto")
    ],
    title="Smart Grocery Cart Assistant",
    description="Get a product list tailored to your dietary
preferences."
)

```

```
if __name__ == "__main__":  
    demo.launch()
```

* Running on local URL: `http://127.0.0.1:7860`

* To create a public link, set ``share=True`` in ``launch()``.

Smart Grocery Cart Assistant

Get a product list tailored to your dietary preferences.

Describe your grocery needs (e.g., 'high protein, no besan or curd')

Model

Llama-3.3 (70B) (Groq) ▼

Temperature

0.7



Clear

Submit

Considerations

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).